

Abstract

Deep Neural Networks (DNNs) are pivotal in numerous applications, spanning image recognition, natural language processing, and increasingly, edge-based inference tasks. Despite their versatility, DNNs present significant computational challenges due to intensive memory and processing demands, particularly for deployment on energy-constrained edge devices. To address these workloads efficiently, specialized neural network accelerators employing diverse dataflow strategies—such as weight-stationary (WS), output-stationary (OS), and input-stationary (IS) —are actively explored in the literature.

An alternative approach, event-driven execution, offers substantial efficiency gains by triggering computation only when new data is available. This method significantly reduces redundant memory accesses and computations by skipping ineffectual operations, which is especially beneficial when inputs arrive asynchronously or exhibit significant sparsity. While established design space exploration frameworks like Timeloop, ZigZag, and Maestro systematically evaluate various dataflows and hardware architectures, they predominantly focus on synchronous and frame-based execution models. These conventional models, including common WS and OS dataflows, assume regular execution patterns and are not inherently designed to exploit the advantages of event-driven paradigms. Consequently, comprehensive support for mapping event-driven execution within current design-space exploration tools is notably lacking.

The absence of dedicated tools capable of capturing the intricacies of event-driven dataflows presents notable research gaps:

1. **Sparse Input Exploitation:** Current frameworks inadequately handle sparsity in input data, missing opportunities for computational and memory efficiency.
2. **Multicore Design-Space Exploration:** The vast design space associated with multicore architectures remains insufficiently explored, especially under event-driven conditions.
3. **Event-Driven Dataflow Exploration:** No unified, systematic exploration framework currently exists that fully captures the complexities and advantages of event-driven execution at an intra-core and inter-core level.

To address these critical gaps, this thesis proposes an end-to-end exploration framework specifically tailored for event-driven accelerators. The developed tool leverages and extends the intra-core mapping capabilities of the ZigZag exploration tool to accurately model and evaluate event-driven dataflows. The primary contributions of this work are summarized as follows:

- **Event-Driven Dataflow Modeling:** Introduction of analytical extensions to the ZigZag framework, facilitating accurate exploration and evaluation of input-stationary intra-core dataflow mappings previously unsupported by standard tools.
- **Design-Space Exploration (DSE) of Event-Driven Accelerators:** Conducting comprehensive case studies to analyze the effects of architectural modifications, such as varying SRAM and Processing Element (PE) configurations. Additionally, evaluating the impact of replacing SRAM with MRAM on latency and energy metrics. The ultimate goal is identifying optimal event-driven architectures that enhance criteria such as energy efficiency, latency, and Energy-Delay Product (EDP) across multiple workloads.

A high-level representation of the proposed contributions is illustrated in Figure 1, depicting the integration of hardware configuration, workload configuration, and mapping into a unified exploration framework. This holistic approach ensures comprehensive coverage of the design space, yielding overall energy and latency graphs, optimized mapping schedules, and resource utilization stats.

By addressing these fundamental gaps, this thesis lays the groundwork for systematically realizing the full potential of event-driven accelerators. Subsequent implementation and evaluations will demonstrate the effectiveness and practicality of the proposed framework, establishing its relevance for future research and practical deployment in energy-constrained, high-performance edge computing scenarios.

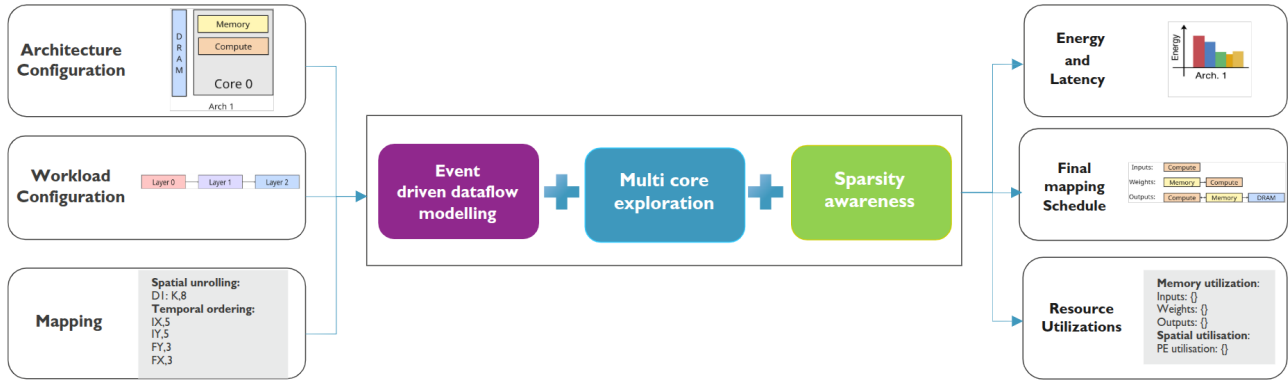


Figure 1: High-level Block diagram